FOUR PRESENTERS / ONE TEN-MINUTE DEMO

WhatMeal Presentation & Code Defense

Group rehearsal pack | **October 29, 2026**

The full preparation checklist, timed script, code-defense guide, feature index and instructor Q&A, combined without removing their technical references or verification limits.

Presenter	Demo slot	Focus
Jabez	00:00-02:30	Profiles, manual plan, calendar, Home
Leonard	02:30-05:00	AI, save/reopen, meal details, favorites
Kyle	05:00-07:30	Meal states, pantry, groceries
Jobert	07:30-10:00	Households, sharing, inbox, closing

Total: 10:00, including handoffs and closing. Q&A and technical study are outside the timed script.

Inside this pack

1. WhatMeal demonstration preparation . . . 2
2. WhatMeal: 10-minute demonstration . . . 6
3. WhatMeal code-defense guide . . . 15
4. WhatMeal feature-to-code index . . . 29
5. WhatMeal instructor Q&A; . . . 34

1. WhatMeal demonstration preparation

For October 29, 2026. This is a preparation checklist, **not a record that two phones have passed**. The coding agent did not access either phone, execute these steps, run new tests or rebuild an APK for this documentation task.

Choose and verify the actual build

- ☐ Confirm the same intended Android APK is installed on both phones. Do not assume a local/showcase/cloud-test APK is the online app.
- ☐ The recorded October 26 online artifact is `build/phone-release/20261026-prototype-gemini-d8c1/WhatMeal-release.apk`, package `com.example.whatmeal`, `productionRelease`, Firebase project `whatmeal-101`. Its recorded SHA-256 is `76060693292D9193D94EBFE245CFF3FEF8A6D20E0A74B39D5FCEBDB00832F623`. This identifies an existing artifact, not a new October 29 build.
- ☐ The current source removes the obsolete production debug-provider override. The October 26 artifact predates that source change but was recorded with the override **false**. Do not say the APK was rebuilt after removal.
- ☐ Install an update without uninstalling or clearing an existing account's data. Do not change phone users. Device actions belong to the presenters; no coding-agent phone access is implied.
- ☐ Native release startup uses Play Integrity; do not switch to a debug provider or change Firebase settings for the talk. `local`/`showcase` use local backends and deliberately block live AI/native push. `aitest`/`cloudtest` are separate debug-only test workflows, not this two-phone release demo.

Accounts and authorization

- ☐ Use two **existing, authorized demonstration accounts**: A is household owner; B is member or admin with editing permission. Do not use the same account on both phones as evidence of two-user authorization.
- ☐ Both accounts are already email-verified and signed in. Test their actual sign-in before the timer. Registration and email delivery are deliberately outside the ten-minute live sequence.
- ☐ Confirm both belong to the same intended household. If joining is necessary, the owner creates one invitation through **Profile → Family Sharing → Create Invitation / Invite Member**. B uses **Join with Key → Invite Code / Key → Join Family Space**. Transfer and redeem it privately before presenting.
- ☐ Check invitation expiry, case-sensitive entry and single use; never include the real invitation in these documents, screenshots shared publicly or source control. Do not demonstrate leaving/rejoining by deleting a working membership during the talk.
- ☐ Confirm both show the correct household, profiles, saved plans, pantry and groceries. Personal records are not automatically copied into a household when joining.
- ☐ View Only cannot perform the planned shared edits. Household account members and planned-for family profiles are different records; do not confuse them.
- ☐ Do not edit, archive, delete, remove members from, or transfer ownership of unrelated households.

Fill a private cue card with real records

No account, password, provider key, recipe, household or ingredient record is invented by this guide. Select actual records with the account owner's agreement.

Needed selection	What the team must record privately
Household and accounts	Real household name; which presenter holds A/B; confirm their roles. Do not write passwords on a public card.
Planned-for profile	Existing profile ID/name, genuine preferences and restrictions, and a cuisine selected in the creator. Do not weaken an allergy to make a generated result pass.
Manual recipe	Actual reviewed dish name and ingredient text, a meaningful plan title, and chosen schedule date. Use one day and Dinner for speed.
Saved backup	Exact existing plan title/date, permission to open it on B, and the meal to inspect. Call it previously generated only if that is actually known.
Grocery example	Accepted meals with interpretable quantities, at least one fully covered ingredient and a separate real shortage if showing both sections. Partial coverage of a single ingredient does not guarantee an AVAILABLE section. If no covered requirement exists, show only the actual section present.
Completion example	An actual accepted saved meal with recorded recipe ingredients and sufficient current pantry coverage. Verify **Mark Completed** → **Confirm** in rehearsal. The pantry check can reject the action after confirmation; do not bypass that rejection.
Shared update item	An agreed noncritical pantry record and two legitimate rehearsed quantity values. Avoid disrupting another person's real inventory or the timed completion example.
Inbox example	A real historical notification to open if the new event is delayed. Distinguish it aloud from new delivery.

AI readiness before the timer

- ☐ On **Profile** → **AI Settings**, verify the actually selected provider and model. Do not show or copy secret fields publicly.
- ☐ The current source default model is `gemini-3.5-flash-lite`. Existing explicitly saved choices are not automatically replaced; confirm the actual selection.
- ☐ In the configured academic build, the existing credential pool uses the Gemini Developer API directly; a personal Gemini key takes priority. If neither is configured, the retained Firebase AI Logic route requires valid App Check. Do not claim every deployment uses the same path.
- ☐ Free-tier availability depends on the provider/project's quota and access. An app limit is not a promise of unlimited free generation. Shared prototype credentials inside an APK are extractable; this is not a secure commercial key-distribution architecture.
- ☐ If permitted, perform a small real generation and **save/reopen it through the installed app** during rehearsal. Record the actual provider/model, success/failure and whether the save completed. A successful **Test Connection** alone is insufficient.
- ☐ Preconfigure **one day, Dinner, a selected planned-for profile, at least one cuisine**. Cuisine is required by validation despite its nearby “Optional” caption. Verify loaded pantry/grocery feeds and synchronized provider settings.
- ☐ B has an accessible saved backup. By **03:25**, switch to B if A has not completed. Do not launch a second request. If only one phone is available or both are busy, omit live generation and say that the shown plan was saved earlier.
- ☐ Do not claim a fresh save if the result is still an in-memory draft. Use **Save Plan**, wait for confirmed success, reopen through **Saved** → **Load Plan**, check date and meal identity.

Connectivity and two-phone rehearsal

- ☐ Use stable internet on both phones for the online Firebase/AI demo. Confirm Wi-Fi/mobile data before starting; configure screen mirroring before the timer.
- ☐ Keep both apps foregrounded for the sharing/inbox check. Native background push is a separate, unverified workflow and is not required here.
- ☐ With B already a member and watching Pantry, A edits the agreed item. Observe the new quantity on B before calling synchronization successful. Do not use A for this step if an earlier request/save is pending or a new draft is unsaved; retain it and label the new check NOT RUN.
- ☐ On B, note the unread/newest-event baseline, then identify the **Pantry** event created after Jobert's shared update and tap it. Earlier pantry edits have the same generic title/body. Confirm read state and routing back to Pantry. A's own pantry event excludes its actor; do not use A's inbox as evidence of fan-out failure.
- ☐ New members do not receive all older household activity: the feed filters against membership `joinedAt`. Create the test update after B has joined.
- ☐ Record **PASS / FAIL / NOT RUN** separately for A launch, B launch, sign-in, manual save/reopen, AI generation, AI save/reopen, calendar date, grocery update, cross-account quantity refresh, and inbox routing. Do not fill PASS from this document.

Timing and recovery

- ☐ One timer runs continuously for 10:00. Jabez stops at02:30, Leonard05:00, Kyle07:30, Jobert10:00, including transitions.
- ☐ Rehearse the actual recipe entry; do not type an entire long recipe live. Preconfigure only the settings allowed by the script; do not silently pre-save the result and call it a new successful write.
- ☐ AI progress is non-dismissible. It has no Cancel button. Provider request timeout does not bound every preceding database/settings operation.
- ☐ If A is stuck, leave it alone and use B's real backup. Do not pretend that closing an error or seeing an editor proves generation or persistence.
- ☐ Failed **or still-pending/unconfirmed** saves require the same timed fallback: at01:45 for Jabez or03:55 for Leonard, leave A's save/editor untouched and use B. Do not select **Discard**. Retry only after the timed demo unless the allotted slot permits one clear recovery.
- ☐ Retry a failed data feed through its actual Retry control; do not repeatedly tap a disabled action, clear data or dismiss system failures to hide them.
- ☐ Missing image: continue with the intentional fallback and recipe text. No photo source guarantees a matching photograph for every dish.
- ☐ If behind, omit extra month navigation or a new manual grocery item. Preserve the four equal time allocations.
- ☐ End at10:00. The Q&A guide is a study resource, not an extra scripted segment.

Evidence and known limitations

Area	Current evidence	What remains unverified/limited
Automated application checks	Recorded October26 full suite: 675 passes, one skip; clean analysis. October28 cleanup:31 focused Flutter passes,6 web/bootstrap passes, analysis of `lib/main.dart` clean.	These do not prove two physical phones, live Firebase listeners, or every provider/model. No tests rerun for these documents.
Authorization	Recorded71-case Firestore emulator acceptance; canonical membership, role/schema cases.	Not a new physical-device or production-data acceptance session.
Live AI/persistence	October26 bounded host proof: five small credential probes, one `gemini-3.5-flash-lite` meal, isolated Spark repository/REST save/reopen.	Not a release-APK UI proof; no claim all five credentials generated full plans or all models/providers work live.
Android	Existing APK build/signature/hash and older emulator workflows. User reports the app is working.	Coding agent performed no new two-phone acceptance. User-observed results must be labeled user-verified.
Notifications	In-app activity/receipt/routing implementation and automated evidence.	Background/cold-start native push delivery and trusted sending infrastructure not fully verified.
Stock behavior	Quantity matching/projection tests and source.	Grocery checks do not add pantry stock; completion does not deduct it; manual duplicate names are not quantity aggregation.
Profile recovery	Account-profile retry and Home recommendation feed recovery implemented.	Profile's family-member list does not itself expose all repository loading/error states. Avoid forced-outage demonstration.
Security	Release Play Integrity retained; Auth and Firestore rules remain distinct controls.	Historical App Check debug credential exposure remains unresolved; the owner explicitly accepted that known Git-history risk for the push. No secrets were revoked by this documentation task.

Detailed prior evidence: `audit/final/ACADEMIC_RELEASE_20261026.md`; current trace and limitations: `CODE_DEFENSE_GUIDE.md`. Do not claim production readiness during an academic presentation.

2. WhatMeal: 10-minute demonstration

Presentation date: **October 29, 2026**. Presenters: **Jabez, Leonard, Kyle, Jobert**.

This is a **10:00 schedule**, including speech, taps, waiting, handoffs and closing. It is not a guarantee that a network request will finish on time. Use a visible timer and the fixed cutoffs below. Do not add an introduction or conclusion outside these slots. Technical questions are prepared in INSTRUCTOR_QA.md; if interrupted, answer briefly and skip the next optional action inside your own slot.

Before the timer

- Use the real Android release on two phones, **A: household owner; B: a different member account**. Both accounts must already be verified, signed in, and members of the same authorized household. Do not show passwords, provider keys or invitation values on the projector.
- Rehearse on the exact installed APK. Actual two-physical-phone acceptance has **not** been performed by the coding agent. Source inspection and automated tests are not a substitute.
- Fill a private cue card with the **actual** household name, selected planned-for profile, manual recipe name and ingredients, chosen date, backup plan name, completion-ready meal, pantry item and grocery item. These documents intentionally supply no invented accounts, credentials or food records.
- Preconfigure the creator for **one day, Dinner, an existing planned-for profile and one cuisine**. Return A to Home. B must have the real, already saved backup plan accessible. Verify its date and ingredients beforehand.
- Jabez's manual dish must use a genuine recipe your team has reviewed. Prepare its real ingredient text so it can be entered quickly. Do not save blank placeholder meals.
- The live AI dialog **cannot be cancelled**. If it is still open at **03:25**, continue on B using the saved backup. Leave A's request alone. Later, an unsaved result or pending save on A is also a reason to skip the live two-phone update; do not discard it just to navigate.
- All documents use current source references. The existing October 26 APK predates removal of the obsolete release debug override, although its recorded build set that override to false. No APK was rebuilt for this documentation task.

Timing summary

Presenter	Start	End	Allocation	Main responsibility
Jabez	00:00	02:30	02:30	Account/profile context, manual creation, calendar and recommendations
Leonard	02:30	05:00	02:30	AI generation, explicit saving/reopening, meal details and favorite
Kyle	05:00	07:30	02:30	Accepted/completed states, pantry and groceries
Jobert	07:30	10:00	02:30	Household roles, two-account sharing, in-app notifications and closing
Total	**00:00**	**10:00**	**10:00**	All transitions included

Jabez - 00:00-02:30

Time	Exact speaking lines	Screen and actions	Feature/code cue
00:00-00:20	"Good day. We are Jabez, Leonard, Kyle and Jobert. WhatMeal connects meal planning, pantry records and household shopping. We will use two prepared accounts."	A starts on **Home**. Point to the five bottom tabs; do not read every label aloud.	`AppShell.build`, `lib/core/widgets/app_shell.dart:57-239`: the stateful navigation shell keeps the tabs.
00:20-00:40	"This account is verified. These profiles describe the people eating and their restrictions; household accounts control who can access shared data."	**Profile** → show **Family Profiles & Dietary Preferences** and one existing profile. Do not change an allergy during the talk.	`createAppRouter`, `lib/core/routing/app_router.dart:29-56`; `FamilyMemberRepository.addMember/updateMember`, `lib/features/profile/data/family_member_repository.dart:124-144`. Authentication gates routes; profiles provide meal context.
00:40-01:10	"Manual planning needs no AI call. I choose the date, people and meal slots, then create a custom plan."	**Meal Plan** → **Create / Generate** if the creator is hidden → **Create Own Plan**. Use the prepared **Plan Title**, profile under **Plan Meals For**, **Number of Days: 1**, and **Include Meal Types: Dinner**. Confirm the selected calendar date. Tap **Create Custom Meal Plan**.	`_createCustomMealPlan`, `lib/features/meal_plan/pages/meal_plan_page.dart:118-183`: creates editable local template slots and a dated draft.
01:10-01:45	"I enter our reviewed dish. It remains a draft until saving succeeds; untouched blank slots are omitted."	In **Meal Plan Editor**, tap the meal's pencil, tooltip **Edit Dish**. In **Edit Meal Details**, enter the cue-card **Dish Name** and **Ingredients (comma-separated)**; tap **Save Changes**. Tap the plan save/bookmark action → **Save Meal Plan**, confirm **Plan Name** → **Save Plan**. Wait for **Meal Plan Saved!** 📡, then **Go to Calendar Now**.	`_showSavePlanDialog`, `_saveDraft`, `lib/features/meal_plan/pages/generated_meal_plan_page.dart:51-96,301-445,473-630`; `GeneratedPlanDraft.mealsForSaving`, `lib/features/meal_plan/models/saved_meal_plan.dart:365-385`. The repository write, not the dialog closing, establishes persistence.
01:45-02:10	"The calendar uses the scheduled date, not the record's creation time."	**Home** → **Calendar**. Select the actual saved date; show the plan. **Only if ahead of time**, tap **Next month**, **Previous month**, then reselect the prepared date if necessary. Do not use **Today** unless the plan is scheduled today.	`_MealPlanCalendarPageState._moveMonth/_select`, `lib/features/meal_plan/pages/meal_plan_calendar_page.dart:39-90`; `calendarPlansForDay`, `lib/features/meal_plan/models/calendar_plan_day.dart:7-43`. Changing months clamps invalid day numbers.
02:10-02:20	"Home ranks suitable saved meals for today. It makes no new AI request."	**Back** → **Home**, then point to **Recommended for Today**. If empty, replace the line with: "No saved meal matches today; Home does not generate one automatically."	`homeRecommendations`, `lib/features/home/models/home_meals.dart:20-77`; `HomePage` recommendation builder, `lib/features/home/pages/home_page.dart:487-573`.

Time	Exact speaking lines	Screen and actions	Feature/code cue
02:20-02:30	“Leonard will now show generation, saving and reopening.”	Hand A to Leonard. If the manual save is unconfirmed, retain its draft/save on A and use B for the prepared-plan portion.	**Handoff included in Jabez's 2:30.**

Jabez cutoff: at01:45 use B whenever save success is **not confirmed**, including a still-pending write. Say, “This save has not been confirmed; we are using our prepared saved plan.” Leave A's editor/save untouched. If B becomes the active phone, Leonard must skip live AI rather than start a blocking request on the only available phone.

Leonard - 02:30-05:00

Time	Exact speaking lines	Screen and actions	Feature/code cue
02:30-02:55	“AI generation uses the selected provider and model. Our configured prototype can call Gemini directly. Personal provider keys are optional; free access still depends on the provider's available quota.”	On A, **Meal Plan → Create / Generate → AI Generator**. Confirm the prepared member, one day, Dinner, one cuisine and **Prioritize Pantry**. The selected provider/model must already have been checked in **Profile → AI Settings** before the timer. Do not open key fields or run **Test Connection** now.	`_generateMealPlan`, `lib/features/meal_plan/pages/meal_plan_page.dart:186-351`; `AiProviderRepository.generationReadinessError`, `lib/features/ai_settings/data/ai_provider_repository.dart:96-113`. The current default is conditional on actual configuration, not always Firebase AI.
02:55-03:05	“I am requesting one dinner to keep the demonstration small.”	Tap **Generate AI Meal Plan once**. Show **Generating AI Meal Plan**.	`_MealPlanPageState._generateMealPlan`; `GenerationInventory.capture`, `lib/features/meal_plan/data/generation_inventory.dart:22-40`. Inventory must be loaded in the current workspace before requesting.
03:05-03:25	“The request includes the selected people, cuisine, restrictions and recorded pantry quantities. The response is parsed and checked for valid meal slots and recipe data. We still review the recipe ourselves.”	While the dialog is visible, explain without tapping again. At **03:25**, use the branch below immediately.	`LlmService.generateMealPlan`, `lib/core/services/llm_service.dart:271-351`; `AiMealPlanRequest.buildPrompt/validateMeals`; `AiJsonParser.parseMealList`.
03:25-03:55	**Success:** “The app reports the provider and model actually used. A generated result is not automatically saved; I must save this plan.” **Fallback:** “The live request has not completed. This is our previously saved plan, not a successful new generation.”	**If ready:** inspect the result → save/bookmark → **Save Meal Plan** → **Plan Name** → **Save Plan**; await success. **If still loading or failed:** show B's actual backup through **Meal Plan → Saved → Load Plan**. Leave A alone. If saving fails, retain the draft on A and use B.	`AiGenerationSnackBar`; `_saveDraft`, `lib/features/meal_plan/pages/generated_meal_plan_page.dart:51-96`; `SavedPlansRepository.savePlan`, `lib/features/meal_plan/data/saved_plans_repository.dart:269-321`. Never force-dismiss the non-dismissible generation dialog.
03:55-04:25	“Reopening the plan reads the saved data. The plan's path stays attached to its meals, so similar meal IDs in another plan do not select the wrong record.”	**At03:55, if save success is still unconfirmed, switch to B's backup and leave A untouched.** After a confirmed save, choose **View All Saved Plans** or **Go to Calendar Now → Saved**. Find the actual plan by its title/date → **Load Plan**. If already using B's backup, reopen that same known plan.	`SavedPlansRepository`; `watchSavedPlans`, `lib/features/meal_plan/data/saved_plan_stream.dart:7-81`; `mealRoute/resolveSavedMeal`, `lib/core/routing/meal_route.dart:19-50`.
04:25-04:50	“Meal details show the ingredients and cooking steps we have stored. Favorites are saved on the exact meal. An unavailable photo shows a fallback instead of preventing us from reading the recipe.”	Open the known recipe card. Show its real ingredient list and instructions. Tap the heart **Add favorite** if it is not already favorited; otherwise point out its saved state. Do not toggle repeatedly.	`_MealDetailPageState._resolveMeal`, `lib/features/meal/pages/meal_detail_page.dart:76-93`; favorite callback at434-468; `SavedPlansRepository.toggleFavorite`,377-391; `_MealNetworkImageState._resolve`.

Time	Exact speaking lines	Screen and actions	Feature/code cue
04:50-05:00	"We have separated generation from saving and reopening. Kyle will now show how accepted meals relate to pantry stock and groceries."	Give the active, responsive phone to Kyle. Leave any unresolved A request/draft untouched.	**Handoff included in Leonard's 2:30.**

AI timing rule: the per-request timeout is not a whole-screen deadline. The modal cannot be cancelled, and configuration/quota operations have no overall timeout. B is therefore required for this timed fallback. If both phones are not ready before presenting, choose **saved-plan explanation mode** before starting and do not launch live AI.

Kyle - 05:00-07:30

Time	Exact speaking lines	Screen and actions	Feature/code cue
05:00-05:20	"Saved meals have persisted states. Accepted meals contribute shopping requirements; a badge is not just a color."	On the saved meal, if **Pending**, tap **Accept Meal** → **Confirm**. If already accepted, show that state without resetting it. Use the preflight-approved meal if the new AI recipe is unsuitable for this step.	`_MealDetailPageState._updateStatus`, `lib/features/meal/pages/meal_detail_page.dart:112-188`; `MealWorkflowStore.setStatus`, `lib/core/services/meal_workflow_store.dart:10-52`.
05:20-05:50	"Pantry items store their recorded quantity, unit text and expiration date. I will update our prepared demonstration item."	**Pantry** → **Item options** → **Edit Item**. In **Edit Ingredient**, change to the actual rehearsed quantity/expiration value → **Save Changes**. Wait for confirmation. Do not edit a person's real stock without agreement.	`_PantryPageState._editItem`,92-132; `PantryRepository.updateItem`; `PantryStore.write`,8-48. It updates the same document and creates shared activity atomically.
05:50-06:20	"Groceries combine accepted meal requirements and subtract usable pantry stock. Unsupported quantities are flagged rather than guessed."	**Grocery**. Show **NEEDED FOR MEALS** and, if present, **AVAILABLE IN PANTRY**, using real displayed quantities. Point to **Check recipe quantity** if an amount cannot be interpreted.	`MealGroceryProjection.calculate/IngredientQuantity`, `lib/features/grocery/data/meal_grocery_projection.dart:54-201`. Compatible unit groups can convert; incompatible units remain separate.
06:20-06:40	"Checking this row records a shopping marker; it does not add pantry stock."	Check one agreed **Needed** row. Show its purchased/Used appearance. Do not check an Available row; those controls are disabled. Avoid **Clear Used** during the presentation.	`GroceryRepository.setPurchased`,343-381; `GroceryItemTile` controls, `lib/features/grocery/pages/grocery_page.dart:400-489`.
06:40-07:00	"Manual shopping items are supported too. Meal-derived requirements must be changed through their meals."	Grocery plus → **Add Grocery Item**; show its fields. **Only if rehearsed and ahead of time**, enter the actual approved shopping item/quantity/aisle → **Add to List**. Otherwise **Cancel**. Use the remaining seconds to navigate **Meal Plan** → **Saved** → **Load Plan** → **the completion-ready meal**. Do not create a duplicate variation merely to force a row.	`GroceryItemEditor`, `lib/features/grocery/widgets/grocery_item_editor.dart:88-169`; `GroceryStore.addUnique`, `lib/features/grocery/data/grocery_store.dart:52-89`. Name normalization prevents new same-name duplicates; it does not sum manual quantities.
07:00-07:20	"Completion requires the pantry check to pass. It records the meal state without deducting stock."	On the prepared accepted, completion-ready meal: **Mark Completed** → **Confirm** once. The pantry check runs after confirmation. If rejected, do not bypass it: show an existing completed meal via **Home** → **Done** and identify it as a previous record.	`mealCookingProblem`, `lib/features/meal/widgets/meal_ingredients_panel.dart:9-26`; `_MealDetailPageState._updateStatus`; `MealWorkflowStore.setStatus`.
07:20-07:30	"These food records can be shared with authorized household accounts. Jobert will show that sharing and the in-app notification history."	Hand the phones to Jobert.	**Handoff included in Kyle's 2:30.**

Jobert - 07:30-10:00

Time	Exact speaking lines	Screen and actions	Feature/code cue
07:30-07:55	"These are different accounts in the same household. We joined with an invitation before presenting. Account membership controls access; planned-for profiles supply meal preferences."	On a responsive phone: **Profile** → **Family Sharing**. Show **Family Sharing Space**, member names and role labels. Do not generate or expose a new invitation during the timed demo.	`FamilyService.createInvitation/joinFamily`, `lib/core/services/family_service.dart:93-170`. Canonical membership authorizes later access. Join is described, not falsely presented as live redemption.
07:55-08:15	"Roles control allowed actions. These household accounts are separate from the people whose meals we plan."	Tap **Open Household Dashboard**. Show **Household Account Members** and **Family Members (Planned For)**. Do not change roles, transfer ownership, remove a member or archive the household.	`HouseholdDashboardPage`, `lib/features/family/pages/household_dashboard_page.dart:134-185`; `FamilyRoleX`, `lib/features/family/models/family_space.dart:8-38`.
08:15-08:40	**If ready:** "We are now checking a real shared update between these accounts." **Otherwise:** "We will show existing shared data; this new two-phone check is not being run."	Proceed only when both phones are responsive **and A has no pending request/save or unsaved draft**. Note B's unread/newest-event baseline; put B on **Pantry**. On A, **Pantry** → **Item options** → **Edit Item** → agreed noncritical quantity → **Save Changes**. Otherwise preserve A and use B's existing shared view.	`PantryStore.write`, `lib/features/pantry/data/pantry_store.dart:8-48`; `PantryRepository._listenToItems`, `lib/features/pantry/data/pantry_repository.dart:103-129`.
08:40-09:00	**If observed:** "The second account now shows the same saved quantity." **If not observed:** "The second phone has not confirmed the update; this live check is unconfirmed."	Compare the actual item on B. Allow only the time in this slot. If A remains blocked by AI or the update does not arrive, show the existing shared view and explain the intended stream path; do not simulate success.	`FamilyRepository._bindAccount`, `lib/features/family/data/family_repository.dart:36-72`; repository `ValueNotifier` → `ValueListenableBuilder`, with scope/session checks.
09:00-09:30	"This is the in-app history. Opening an event marks it read for this account; other members keep their own read states."	On B: bell → **Notifications & Alerts**. Identify the **newest Pantry event after the08:15 update**, using its time/new unread arrival, with body **A household member updated pantry stock.** Tap it and show return to Pantry. Kyle's earlier edit can have the same title/body; do not misidentify that old event as Jobert's new delivery. If no new card arrived, label any opened existing card as historical.	`HouseholdActivityStore.watch/markRead`, `lib/core/services/household_activity_store.dart:50-174`; `NotificationsPage` onOpen, `lib/features/notifications/pages/notifications_page.dart:18-45`; `notificationRoute`, `lib/core/routing/notification_route.dart:6-74`.
09:30-09:45	"These are in-app alerts. Background push and production readiness are not claimed. Authentication, rules and App Check remain separate controls."	Keep the final relevant screen visible. No system settings, permission resets or secret fields.	`NotificationService.initialize`, `lib/core/services/notification_service.dart:243-395`; release Play Integrity in `lib/main.dart:157-160`.

Time	Exact speaking lines	Screen and actions	Feature/code cue
09:45-10:00	"WhatMeal connects meal planning, pantry records, groceries and household coordination in one Flutter app. We have shown our successful actions and clearly identified any fallback. Thank you."	Stop the timer at **10:00**.	**Closing included in Jobert's 2:30. No extra closing segment.**

Fixed fallback rules

- **Slow AI at 03:25:** switch the displayed workflow to B. Say it is a previously saved plan. Do not retry, force-close, tap Back through the modal, or claim a fresh result.
- **Save failure:** say the result is not confirmed saved; keep the draft open. Use B's already saved plan. Never choose **Discard** to hide a failure.
- **No recommendation/photo:** show the actual empty state or intentional image fallback. Neither establishes an AI failure or successful image match.
- **Completion blocked:** show a real existing completed record and state that it was completed earlier. Do not invent pantry stock or disable checks.
- **No shared update/inbox event by the slot end:** explicitly mark that live step unconfirmed and continue with a real existing view.
- If a presenter is behind, omit a month-navigation tap or the optional new grocery row. Keep all four 2:30 boundaries. Do not speed-read the entire code guide.

3. WhatMeal code-defense guide

Prepared from the current source on **October 29, 2026**. Paths are relative to the repository root; line ranges refer to this inspected version and may move after edits. These are explanations to study, not additional speech outside the ten-minute script.

No runtime source, Firebase settings, Git index, APK or phone was changed for this task. No new automated, live-provider or physical-device tests were run. Use the evidence table in DEMO_PREPARATION_CHECKLIST.md when discussing verification.

Architecture answer every presenter should know

> "WhatMeal separates Flutter screens from repositories and services. GoRouter manages navigation and authentication redirects. Repositories listen to Firebase and expose state through ChangeNotifier or ValueNotifier, so widgets rebuild when data changes. Models translate stored data into Dart objects. AI requests pass through a provider adapter, parser and validator, and the user explicitly saves the resulting plan."

```
Tap / form input
  → Page callback and validation
  → Repository or service
  → Firebase Auth / Firestore, or an AI adapter and HTTP/SDK call
  → Model parsing and validation
  → ValueNotifier / ChangeNotifier notification
  → ValueListenableBuilder / router / widget rebuild
```

The architecture is not uniformly "Provider does everything." `lib/main.dart:214-220` injects services and a `ChangeNotifierProvider<MealPlanRepository>`; many active screens use singleton `ValueNotifier` repositories directly. Local form state uses `setState`. `lib/app.dart:8-59` owns the router; `lib/core/routing/app_router.dart:29-178` defines auth redirects and the five indexed-stack branches. Firestore snapshot listeners are not the same as polling or sending push notifications.

Shared data map

Purpose	Confirmed path
Account profile	`users/{uid}`
Planned-for people	`users/{uid}/familyMembers/{profileId}` or `families/{familyId}/familyMembers/{profileId}`
Pantry	`users/{uid}/pantryItems/{itemId}` or `families/{familyId}/pantryItems/{itemId}`
Grocery rows	`{scope}/{id}/groceryLists/{listId}/items/{itemId}`, where scope is `users` or `families`
Plan and recipes	`{scope}/{id}/mealPlans/{planId}` and its `meals/{mealId}` subcollection
Canonical household membership	`families/{familyId}/members/{uid}` and `membershipIndex/{uid}`
Household invitation	`householdInvitations/{token}`; never show an actual token in documentation
Shared activity	`families/{familyId}/activity/{eventId}`
Per-account shared read state	`users/{uid}/notificationReceipts/household:{familyId}:{eventId}`
Personal notifications	`users/{uid}/notifications/{notificationId}`
AI selection/credentials	`users/{uid}/secrets/llmConfig`; owner-scoped storage is not an OS secure-key vault
Cooperative app quota	`users/{uid}/usageCounters/daily`

Jabez: account context, manual planning and Home

J1. Authentication and navigation

- **Files/symbols:** `lib/features/auth/pages/login_page.dart:27-59,239-287`, `_LoginPageState._handleLogin`; `lib/features/auth/data/auth_repository.dart:83-175,198-254,326-371`, `AuthRepository.login/register/reloadUser`; `lib/features/auth/pages/verify_email_page.dart:39-98,194-257`, `_checkVerification/_resendVerificationEmail`; `lib/core/routing/app_router.dart:29-104`, `createAppRouter`; `lib/features/auth/models/user_model.dart:1-35`, `UserModel`.
- **Trigger/flow: Sign In** validates form → `login` → Firebase Auth sign-in → merges account metadata into `users/{uid}`. **Create Account** checks form/terms → Auth account creation → verification attempt/profile write. Router redirects unsigned users to login and unverified users to verification.
- **Validation/data:** Auth identifies the account by UID, not a typed household name. Login preserves an existing edited display name. Passwords go to Firebase Auth, never to the Firestore profile document.
- **Loading/failure:** Login/register disable their buttons while awaiting. Error codes map to safe messages. Verification can require a resend; registration success does not prove email delivery. Auth profile snapshot callbacks check session/UID before applying state.
- **Say aloud, about 20 seconds:** "The login page calls our authentication repository. Firebase Auth returns the account identity, and the repository synchronizes profile metadata. GoRouter listens to that repository and sends an unsigned user to login or an unverified user to email verification. The screen does not grant its own database permission."
- **Likely question:** Why not just hide protected tabs? **Answer:** Tabs are a UI concern; the router gates navigation and Firestore rules separately authorize data access.

J2. Planned-for profiles, preferences and restrictions

- **Files/symbols:** `lib/features/profile/pages/profile_page.dart:20-94,325-346,466-483`, `ProfilePage._openAddMemberSheet/_openEditMemberSheet`; `lib/features/profile/widgets/add_edit_family_member_sheet.dart:118-165,205-458`, `_AddEditFamilyMemberSheetState._submit`; `lib/features/profile/data/family_member_repository.dart:59-144`, `FamilyMemberRepository.addMember/updateMember`; `lib/features/profile/models/family_member.dart:3-69`, `FamilyMember`.
- **Trigger/flow: Profile** → **Add** or **Member actions** → **Edit Profile** → sheet validates → constructs model → awaited repository write → Firestore snapshot → profile/Home rebuild.
- **Validation/data:** Name is required and bounded; age is constrained to0-130. Preferences and restrictions are stored with the planned-for person in the current personal/household `familyMembers` collection. This person does not automatically receive an Auth account or household permission.
- **Loading/failure:** Sheet prevents duplicate submission and keeps input on failed save. The repository exposes loading/error/retry. Important limitation: the Profile member list does not consume all those loading/error fields; Home recommendations does. Do not promise perfect recovery on every profile screen.
- **Say aloud:** "These profiles describe the people eating, not the accounts signing in. The form validates the name and age, then saves preferences and restrictions through the profile repository. The active workspace determines whether that record is personal or shared. AI and Home recommendations read this context."
- **Question:** Does adding a child profile invite that child? **Answer:** No. Planned-for profiles and authenticated household members are separate collections and responsibilities.

J3. Manual meal-plan creation and explicit saving

- **Files/symbols:** `lib/features/meal_plan/pages/meal_plan_page.dart:118-183,517-644,1044-1072`, `_MealPlanPageState._createCustomMealPlan`; `lib/features/meal_plan/pages/generated_meal_plan_page.dart:51-96,301-445,473-630`, `_saveDraft/_showSavePlanDialog/_showEditMealDialog`; `lib/features/meal_plan/models/saved_meal_plan.dart:365-385`, `GeneratedPlanDraft.mealsForSaving`; `lib/features/meal_plan/data/saved_plans_repository.dart:269-321`, `SavedPlansRepository.savePlan`; `lib/features/meal_plan/models/meal_plan.dart:3-80`, `MealPlan`.
- **Trigger/flow: Create Own Plan → Create Custom Meal Plan** validates people/types → creates blank local template slots with the selected start date → editor. **Edit Dish → Save Changes** updates a local unsaved meal. **Save Plan** creates the plan/meal documents in an awaited batch.
- **Validation/data:** Untouched blank manual slots are omitted by `mealsForSaving`. A partially entered unnamed meal is invalid. The repository selects the current authorized household path, or the user's personal path when there is no household. It does not silently offer a private-vs-shared save toggle.
- **Loading/failure:** A save is not considered confirmed until the repository finishes; the editor retains a failed draft. **Back** can offer **Keep editing / Discard / Save Plan**. Do not discard a result to hide failure.
- **Say aloud:** "Manual planning creates a draft without an AI call. We edit an actual dish, and untouched blank slots are left out of the saved meal list. Saving writes the plan metadata and its meals through the repository. Until that write succeeds, the editor is only an unsaved draft."
- **Question:** Why did empty template slots once prevent saving? **Answer:** Template placeholders must not be treated as authored recipes. The current save selection omits untouched slots but still rejects partially entered meals without names.

J4. Calendar, dates and status shortcuts

- **Files/symbols:** `lib/features/meal_plan/pages/meal_plan_calendar_page.dart:39-90,98-188,232-267`, `_MealPlanCalendarPageState._select/_moveMonth`; `lib/features/meal_plan/models/calendar_plan_day.dart:7-46`, `calendarPlansForDay/mealCalendarMonth`; `lib/features/meal_plan/models/saved_meal_plan.dart:429-464`, `SavedMealPlan.dateForMeal/activeMeals`; `lib/core/routing/meal_route.dart:3-24`, `mealOverviewRoute/mealCalendarRoute/mealRoute`; `lib/features/home/pages/home_page.dart:252-320,579-589`, `_weekStat`.
- **Trigger/flow: Home → Calendar**, date tap, month arrows → selected/month state → filters current saved plans by scheduled day → opens plan or meal with its identity. **Pending / Accepted / Done / Favorites** opens filtered history for the displayed week.
- **Validation/data:** Meal date derives from the plan's start date plus meal day index. Creation timestamp is not the calendar date. Multiple plans can contribute meals to one day; replaced predecessors are excluded from active views. Month movement clamps a selected day that does not exist in the destination month.
- **Loading/failure:** Saved-plan listener state controls loading and **Retry Calendar**. Undated plans receive an explicit notice. Session checks reject navigation based on an old account snapshot.
- **Say aloud:** "The calendar displays saved plans on their scheduled dates. It calculates each meal's date from the plan start date and day index, and it can show overlapping plans. Month navigation keeps a valid selected day. Opening a meal carries its plan path, so we do not guess which saved plan owns it."
- **Question:** Are Sunday and Monday indexes the same as Dart weekday values? **Answer:** No; `WeekCalendar.dayIndexOf` converts Dart's weekday to a Monday-based zero index. The stored plan's actual start date still determines its meal dates.

J5. Home recommendations and meal-image behavior

- **Files/symbols:** `lib/features/home/models/home_meals.dart:9-77`, `matchesMealFilter/homeRecommendations`; `lib/features/home/pages/home_page.dart:487-573`, Home recommendation builder; `lib/core/services/unsplash_service.dart:76-150,231-270,311-380`, `UnsplashService.validImageUrl/fetchMealImage/dishMatches/_fetchMealDb/_fetchCommons`; `lib/features/meal/widgets/meal_network_image.dart:32-143`, `_MealNetworkImageState._resolve`.
- **Trigger/flow:** Home observes saved plans and planned-for profiles → filters today's pending/accepted recipes → validates restrictions and ranks favorites/preferences → displays up to four. Meal image widget resolves using the actual dish name, not an arbitrary food seed.
- **Validation/data:** Recommendations are local filtering/ranking of authorized saved recipes, not an automatic AI generation. Unresolved participants, missing ingredients, wrong dates and disallowed restrictions exclude a meal. Images can use TheMealDB, Commons and optional configured Unsplash, with title/URL checks and caching.
- **Loading/failure:** Recommendation feeds have spinner, empty state and **Retry Recommendations**. Photo misses/failures use the intentional fallback, with bounded retry/cache behavior. Text matching cannot prove every image's visual correctness or every dietary synonym.
- **Say aloud:** “Home recommends from meals already saved for today. It checks current participants and restrictions, then prioritizes favorites and matching preferences. This avoids an AI request whenever Home rebuilds. Photos are resolved separately by dish identity, and a missing photo does not prevent the recipe from opening.”
- **Question:** Is Recommended for Today a machine-learning recommender? **Answer:** The current Home ranking is rule-based. The separate generation feature calls an external AI provider.

Leonard: AI and saved-plan integrity

L1. Provider/model selection and the actual default route

- **Files/symbols:** `lib/features/ai_settings/pages/ai_settings_page.dart:309-368,485-584,790`, `AiSettingsPage`; `lib/features/ai_settings/data/ai_provider_repository.dart:96-113,166-223,268-322,493-519`, `AiProviderRepository.generationReadinessError/selectProvider/updateModel/removeKey`; `lib/core/services/llm_service.dart:128-179`, `LlmService.getLlmConfig/saveLlmConfig`; `lib/core/services/ai/ai_provider_factory.dart:15-72`, `AiProviderFactory.create/validate`; `lib/core/services/ai/adapters/gemini_adapter.dart:43-76,164-193`, `GeminiAdapter.usesFirebaseAi/_generate`; `lib/core/services/ai/model_registry.dart:45-47`, default model constant.
- **Trigger/flow:** Provider/model selection updates the settings notifier → serialized persisted configuration → generation rereads configuration → validates model/protocol → constructs adapter.
- **Validation/data:** Personal Gemini key takes priority, then the configured academic pool uses direct Gemini Developer API. With neither, the retained Firebase AI Logic path uses Auth/App Check. Config lives at `users/{uid}/secrets/llmConfig`; only the owner should read it under rules. A secret-free clone does not receive the private pool.
- **Loading/failure:** Signed-out, pending-write or unsynchronized settings block generation. Removing credentials restores default behavior rather than requiring a personal key. Key storage is not an OS secure vault; APK-shipped shared credentials are extractable.
- **Say aloud:** "The selected provider and model are persisted per account and validated before a request. In this configured prototype, default Gemini can use the direct Developer API. A personal key overrides it, while the Firebase AI route remains available without the configured pool. We report which provider and model actually answered."
- **Question:** Is generation guaranteed free forever? **Answer:** No. Provider access and quota apply; the app's default configuration does not remove those limits.

L2. Context, inventory readiness and request quota

- **Files/symbols:** `lib/features/meal_plan/pages/meal_plan_page.dart:186-351`, `_generateMealPlan`; `lib/features/meal_plan/data/generation_inventory.dart:22-40`, `GenerationInventory.capture`; `lib/core/services/ai/pantry_prompt_context.dart:5-18`, `pantryPromptContext`; `lib/core/services/llm_service.dart:58,271-351`, `kDailyQuotaLimit/LlmService.generateMealPlan`; `lib/core/services/ai_quota_store.dart:10-41`, `AiQuotaStore`.
- **Trigger/flow: Generate AI Meal Plan** checks editing role, selected members/types/cuisine, UID and workspace → captures loaded pantry/grocery snapshots → reads provider configuration → reserves applicable quota → calls adapter.
- **Validation/data:** Prompt includes recorded member names/ages, restrictions, preferences, cuisine, budget, meal types and inventory quantities. Grocery shopping is described as not yet available stock. Used/needed/expired pantry stock is excluded. The default cooperative quota is10 per account/day; failures after reservation can still consume a reservation.
- **Loading/failure:** Unsynchronized inventory stops before the provider request, rather than pretending the pantry is empty. The progress dialog is non-dismissible. No whole-operation timeout covers all preliminary database work.
- **Say aloud:** "Before generating, the page captures the right household's loaded inventory and selected people. It passes actual quantities and restrictions into the request. The service reserves its applicable daily request allowance before dispatch, which reduces duplicate spending. A failed or ambiguous request is not automatically refunded."
- **Question:** Is this daily counter strong server-side abuse protection? **Answer:** No. It is a cooperative app limit; a client-owned counter and APK-shipped credentials are not a secure public anti-abuse architecture.

L3. Adapter request, JSON parsing and safe failure

- **Files/symbols:** `lib/core/services/ai/adapters/gemini_adapter.dart:108-119,149-193,251-299`, `GeminiAdapter.generateMealPlan/_withBundledKey/_generate`; `lib/core/services/ai/ai_models.dart:38-158,349-358`, `AiMealPlanRequest.validateMeals/buildPrompt`, `AiMealResponse.validateRecipe`; `lib/core/services/ai/ai_json_parser.dart:12-80`, `AiJsonParser.parseMealList`; `lib/core/services/ai/bundled_key_pool.dart:37-53`, `BundledKeyPool.requestKeys/canTryNext`; `lib/core/services/ai/generation_failure.dart:303-304`, timeout message.
- **Trigger/flow:** Adapter builds selected-model request → REST or retained SDK route → extracts text → parser normalizes supported JSON wrappers → validates slot count/types/days, quantities, recipe content and restrictions → service returns meals and provider/model metadata.
- **Validation/data:** Malformed or incomplete recipes do not become successful plans merely because HTTP returned200. The configured pool preserves the selected model and makes bounded credential attempts. Only credential/permission/rate-limit cases qualify to try another credential; network, timeout and malformed output do not.
- **Loading/failure:** Provider HTTP generation has a90-second per-request timeout; this is not a full-screen deadline. Failure maps to sanitized **Generation failed** actions, not a raw key-containing exception. Custom-provider fallback is reported rather than silently claimed as the originally selected provider.
- **Say aloud:** “An adapter handles the provider's request and response format. Our parser then extracts meal JSON, but parsing alone is not enough: validation checks the requested slots and required recipe fields. Errors are classified before display. Retrying is bounded and preserves the model, so we do not secretly claim another model generated the result.”
- **Question:** Does valid JSON mean an allergy-safe recipe? **Answer:** No. Structural and ingredient-name checks reduce invalid output, but they are not a medical, cross-contamination or nutritional certification.

L4. Saving, reopening and exact meal identity

- **Files/symbols:** `lib/features/meal_plan/pages/meal_plan_page.dart:353-412`, draft navigation; `lib/features/meal_plan/pages/generated_meal_plan_page.dart:51-96,301-445`, `_ensurePersisted/_saveDraft/_showSavePlanDialog`; `lib/features/meal_plan/data/saved_plans_repository.dart:86-193,269-321`, `SavedPlansRepository.savePlan/_planRef`; `lib/features/meal_plan/data/saved_plan_stream.dart:7-81`, `watchSavedPlans`; `lib/core/routing/meal_route.dart:19-51`, `mealRoute/resolveSavedMeal`.
- **Trigger/flow:** Generation success constructs `GeneratedPlanDraft` → user **Save Plan** → awaited metadata/recipe batch → saved path assigned → **Saved** → **Load Plan** reads streamed data and reopens the same plan.
- **Validation/data:** Plan lives under current `families/{familyId}/mealPlans/{planId}`, otherwise personal `users/{uid}/mealPlans/{planId}`; meals are nested. Reopening/favorites/replacements carry full `sourcePath` plus meal ID. Session checks reject stale account results and ambiguous IDs.
- **Loading/failure:** Parent metadata and nested meals each have listeners, so child edits can refresh independently. Save failures retain the draft and must not report success. `persistGeneratedPlanDraft` is a defined helper, not the current live UI path; do not describe generation as automatic persistence.
- **Say aloud:** “Generation creates a draft, and saving is a separate database operation. A batch writes plan metadata and nested recipes, then the editor stores the confirmed document path. Reopening uses that path with the meal ID and current session, preventing a same-ID meal in another plan from being selected accidentally.”
- **Question:** Can Firestore failure mean AI also failed? **Answer:** They are separate stages. A successfully generated draft can remain available for a save retry without another AI request.

L5. Favorites, history, reuse and replacement

- **Files/symbols:** `lib/features/meal/pages/meal_detail_page.dart:245-394,434-468`, `_replaceWithAi` and favorite callback; `lib/features/meal_plan/data/saved_plans_repository.dart:377-418`, `toggleFavorite/replaceMeal`; `lib/core/services/meal_replacement_store.dart:17-90`, `MealReplacementStore.replace`; `lib/features/meal_plan/models/saved_meal_plan.dart:290-346,463-464`, reuse transformation and `activeMeals`; `lib/features/meal_plan/pages/meal_history_page.dart:24-69`, `MealHistoryPage` filtering.
- **Trigger/flow:** Heart writes `isFavorite` on exact recipe. History derives filtered saved active meals. **Reuse** creates a new unsaved draft with new IDs/pending states. **Replace with AI** confirms, requests a successor and transacts the replacement links.
- **Validation/data:** Replacement keeps the predecessor, marks it replaced, links it to one successor in the same slot, and hides it from active views. This is not deleting an unrelated plan. Reuse starts a new current-Monday copy; it is not the same as reopening a saved plan with its original date.
- **Loading/failure:** Replacement shows **Asking AI…** and preserves the original if the request/write fails. The transactional store protects against concurrent duplicate successors. Do not demonstrate another slow replacement request inside the timed script.
- **Say aloud:** “Favorites are persisted recipe fields, while history is a view over saved meal states. Reuse makes a new draft and new identities. Replacement is different: it retains the old record and links a single new meal in the same slot through a transaction, which prevents competing replacements from producing two active successors.”
- **Question:** Why not simply overwrite the old meal? **Answer:** Preserving/linking the predecessor supports consistent history, identity and concurrent replacement handling.

Kyle: stock, shopping and meal states

K1. Accepted, completed and not-completed states

- **Files/symbols:** `lib/features/meal/models/meal.dart:17-60`, `MealStatus`; `lib/features/meal/pages/meal_detail_page.dart:112-188,984-1132`, `_updateStatus` and action controls; `lib/features/meal/widgets/meal_ingredients_panel.dart:9-26`, `mealCookingProblem`; `lib/features/meal_plan/data/saved_plans_repository.dart:352-374`, `updateMealStatus`; `lib/core/services/meal_workflow_store.dart:10-52`, `MealWorkflowStore.setStatus`.
- **Trigger/flow: Accept Meal / Mark Completed / Not Completed** → confirmation → current scope/permission/inventory checks → status transaction → plan listener refresh.
- **Validation/data:** Completion checks pantry readiness, ingredient presence and supported quantity coverage. The transaction updates recipe state/accepted compatibility field, plan timestamp and relevant household activity. It does **not** decrement pantry inventory.
- **Loading/failure:** Confirmation precedes the write, and the transaction treats an already-current status as a no-op. The detail-page callback has no dedicated in-flight/busy guard; tap once and await the result. Its pantry check runs after confirmation and can reject completion with feedback. Stale sessions cannot update a newly switched account's meal. Do not claim that missing stock necessarily disables the button beforehand.
- **Say aloud:** "These labels represent persisted meal states. The details page confirms important changes, checks the current account and pantry, then asks the repository to update through a transaction. Completion records that state, but the current implementation does not automatically consume or deduct inventory."
- **Question:** Is a checked grocery item the same as a completed meal? **Answer:** No. Shopping markers, pantry stock and meal lifecycle are separate records.

K2. Pantry CRUD, expiry and same-record editing

- **Files/symbols:** `lib/core/widgets/app_shell.dart:29-50`, `AppShell._fabForTab`; `lib/features/pantry/widgets/add_pantry_item_sheet.dart:68-118,146-237`, `AddPantryItemSheet`; `lib/features/pantry/pages/pantry_page.dart:18-179,490-566`, `_editItem/_confirmDeleteItem`; `lib/features/pantry/data/pantry_repository.dart:56-167`, `PantryRepository.addItem/updateItem/retry`; `lib/features/pantry/data/pantry_store.dart:8-48`, `PantryStore.write`; `lib/features/pantry/models/pantry_item.dart:5-46`, `PantryItem/PantryStatus`.
- **Trigger/flow: Pantry** → **Add pantry item** → **Add Ingredient** → **Add to Pantry**, or **Item options** → **Edit Item** → **Save Changes** → awaited repository/store → current scoped pantry document → snapshot refresh.
- **Validation/data:** Name, quantity text and expiration date are required by the form. Editing updates the same item ID even when category changes. `expiresInDays` is computed from the stored date; the screen highlights/filter-matches items at five days or less.
- **Loading/failure:** Save-in-progress blocks duplicate form submission; failed save keeps input. Delete requires confirmation. Displaying an expiration warning does not prove a scheduled notification service exists.
- **Say aloud:** "A pantry item is one stored record with a name, quantity text, category and expiration date. Editing keeps its identity rather than making a duplicate in another category. The page listens for changes and calculates the expiration warning from the recorded date. A failed save stays visible for correction."
- **Question:** Does the quantity field guarantee every unit can be calculated? **Answer:** No. It stores text; the grocery projection separately parses supported units and shows uncertainty for unsupported amounts.

K3. Pantry-aware grocery requirements and provenance

- **Files/symbols:** `lib/features/grocery/data/meal_grocery_projection.dart:24-50,54-201`, `MealGroceryRequirement.toItem`, `IngredientQuantity`, `MealGroceryProjection.calculate`; `lib/features/grocery/data/grocery_repository.dart:102-166`, dependency readiness and `_publish`; `lib/features/meal_plan/models/saved_meal_plan.dart:463-464`, `activeMeals`; `lib/core/utils/ingredient_matcher.dart`, `IngredientMatcher`.
- **Trigger/flow:** Current saved accepted meals plus loaded pantry → normalize ingredient/compatible unit → sum requirements → subtract usable stock → merge with persisted grocery rows/receipts → notify UI.
- **Validation/data:** It tracks full source meal paths, not a global dish name. Expired, Used and Needed pantry items do not supply stock. Supported conversions include kilograms/grams and liters/milliliters; it does not guess mass-to-volume or package conversions. Different units can remain separate requirements.
- **Loading/failure:** A missing/loading/error dependency must not be treated as an empty pantry and create misleading shortages. Unknown quantities show **Check recipe quantity**. Removing one meal must retain quantities still required by other source meals.
- **Say aloud:** “The grocery list is partly derived from accepted saved meals. It combines compatible ingredient requirements, keeps their source meal paths and subtracts usable pantry quantities. This lets it preserve another meal's requirement when one meal changes. When a quantity cannot be safely interpreted, the app asks us to check the recipe.”
- **Question:** Why include the source plan path? **Answer:** Meal IDs can repeat across plans; provenance must identify the exact stored recipe and its surviving requirement.

K4. Manual groceries, purchase markers and duplicate prevention

- **Files/symbols:** `lib/features/grocery/widgets/grocery_item_editor.dart:88-169`, `GroceryItemEditor`; `lib/features/grocery/data/grocery_store.dart:9-89`, `GroceryStore.normalizedName/updateManual/initialize/addUnique`; `lib/features/grocery/data/grocery_repository.dart:304-381,479-525`, `cycleStatus/setPurchased`; `lib/features/grocery/pages/grocery_page.dart:103-142,400-489`, `_shareGroceryList` and tile controls.
- **Trigger/flow:** Plus → **Add Grocery Item** → **Add to List** → normalized-name transaction. Checkbox → purchase marker/receipt write → UI Used/checked state. **Share List** copies text to clipboard; it does not send a message or invite an account.
- **Validation/data:** New manual rows use deterministic name IDs and a transaction. Case/whitespace duplicates are prevented for current writers, but quantities are not automatically summed for duplicate manual names. Older random-ID duplicates are retained. Meal-derived edits/deletes must be done through their source meals.
- **Loading/failure:** Duplicate/persistence errors are surfaced; scope changes reject stale writes. Purchase receipts consider source paths/covered quantity. **Clear Used** removes eligible checked manual rows, not all meal provenance.
- **Say aloud:** “Manual shopping items are normalized and added transactionally so two current clients do not create the same normalized name twice. Checking a row stores a shopping marker. It does not add pantry stock. Meal-derived rows have source links and receipts, so we do not casually delete requirements that another recipe still needs.”
- **Question:** Does Share List synchronize accounts? **Answer:** No; it copies text. Household synchronization comes from shared Firestore paths and listeners.

K5. Feed readiness, retry and failure recovery

- **Files/symbols:** `lib/features/pantry/data/pantry_repository.dart:56-129`, `retry/_resetListener/_listenToItems`; `lib/features/grocery/data/grocery_repository.dart:102-129,188-298`, `retry/_resetListener/_initializeActiveList`; `lib/features/meal_plan/data/saved_plan_stream.dart:7-81`, `watchSavedPlans`; `lib/core/widgets/listener_recovery_notice.dart`, `ListenerRecoveryNotice`.
- **Trigger/flow:** Auth/household changes reset scoped listeners; stream events update notifier state; terminal errors expose recovery; **Retry** creates new subscriptions for the current scope.
- **Validation/data:** Session/scope epochs prevent a delayed callback from populating the next account's screen. Loading/error is different from an authoritative empty collection. Grocery initialization reuses an existing latest list or transactionally creates `default` when necessary.
- **Loading/failure:** Do not claim unrestricted offline operation. Cached reads may remain available, but authentication, AI, synchronized writes and definitive recovery depend on the actual service state. Retry does not bypass a permission denial.
- **Say aloud:** “A listener can fail permanently, so the repository exposes an error and can attach a fresh stream when Retry is requested. It also records the account or workspace version. That prevents a delayed operation from a previous account from becoming the current account's data, and it prevents us from calculating against an inventory that has not loaded.”
- **Question:** Why not convert every listener error into an empty list? **Answer:** That would confuse denied/offline data with real absence and could create incorrect grocery or user-visible results.

Jobert: household boundaries and notifications

O1. Invitations and canonical membership

- **Files/symbols:** `lib/features/profile/pages/profile_page.dart:277-294`, Family Sharing entry; `lib/features/family/pages/family_space_page.dart:137-148,700-805`, `_issueInvitation`; `lib/features/family/widgets/household_access_sheet.dart:129-205`, `HouseholdAccessSheet`; `lib/features/family/data/family_repository.dart:262-288,290-326`, `createSpace/joinSpaceWithKey/revokeToken`; `lib/core/services/family_service.dart:37-46,60-170`, `FamilyService.createFamily/createInvitation/validateToken/joinFamily/revokeInvitation`.
- **Trigger/flow:** Authorized owner/admin creates a random invitation → invitation document → second authenticated account submits **Join Family Space** → transaction validates and consumes invitation while creating membership and index.
- **Validation/data:** Default invitation is single use, case-sensitive,32 characters and expires after48 hours. Canonical membership is `families/{familyId}/members/{uid}`, located through `membershipIndex/{uid}`; knowing a household ID is insufficient to self-join.
- **Loading/failure:** Repository `_perform` surfaces failure; expired/revoked/used invitations are rejected. An already-member account cannot simply join another household through this form. Do not remove a working membership during the timed demo.
- **Say aloud:** “The invitation is a temporary authorization to join, not the household's public ID. A transaction consumes it and creates the member record and account index together. Subsequent access comes from stored membership and role checks. We pre-joined the presentation accounts to avoid transferring a secret invitation while the timer is running.”
- **Question:** Can an outsider add their UID to a family document? **Answer:** The coordinated membership/rules design rejects direct self-joining; the invitation transaction and allowed schema must agree.

O2. Roles, departure and archival behavior

- **Files/symbols:** `lib/features/family/models/family_space.dart:1-40`, `FamilyRole/FamilyRoleX`; `lib/core/services/family_service.dart:172-236`, `leaveFamily/removeMember/updateMemberRole/deleteFamily/offerOwnership/acceptOwnership`; `lib/features/family/data/family_repository.dart:335-408`, role/removal/departure operations; `firestore.rules:10-16,67-177,207-241`, stored membership, coordinated changes and data-scope authorization.
- **Trigger/flow:** Role-sensitive controls → repository → constrained service write/batch/transaction → rules validate actual stored authority and coordinated data → listeners update.
- **Validation/data:** Owner/admin/member can perform permitted food writes; View Only cannot. Owner/admin management is further constrained by backend role rules. Owner changes require an offered and accepted transfer, not assigning a role string `owner`. Removing/leaving deletes the membership/index pair. `deleteFamily` archives household state and retains nested data; it is not recursive deletion.
- **Loading/failure:** Confirmation dialogs protect significant actions; errors do not grant extra permission. Legacy/inaccessible households can have special notices/recovery, not a permission bypass. Do not exercise destructive/member actions live for the ten-minute talk.
- **Say aloud:** “Roles come from stored membership. The UI reflects permissions, but the security rules still decide whether a write is allowed. Ownership uses a separate transfer flow. Leaving removes the membership relationship, while the household removal operation is archival, retaining nested data instead of pretending a client can safely erase everyone's records.”
- **Question:** Does App Check replace household rules? **Answer:** No. App Check attests the app; Auth supplies identity; rules authorize that identity's data operations.

O3. Two-account sharing and scope switching

- **Files/symbols:** `lib/features/family/data/family_repository.dart:36-72,145-238`, `_bindAccount/_listenToFamily/_listenToMembers/retry`; `lib/features/pantry/data/pantry_repository.dart:62-129`, `scopePath/_listenToItems`; `lib/features/grocery/data/grocery_repository.dart:136-166,177-286`, scope and active-list listeners; `lib/features/meal_plan/data/saved_plans_repository.dart:86-181`, plan feed/session reset.
- **Trigger/flow:** Auth identity → membership index → current household and member stream → scoped food listeners → notifier → each signed-in phone's widgets.
- **Validation/data:** Same-household accounts share paths under `families/{id}`. Personal data stays under `users/{uid}`; joining is not a migration/copy of personal documents. Revoked membership changes the authorized scope. Account changes cancel old listeners and advance epochs.
- **Loading/failure:** Firestore source code supports real-time updates; it is not proof that two physical phones synchronized during this documentation task. Demonstrate a real observed update or state that it was not confirmed. Never claim a delayed cached display proves server synchronization.
- **Say aloud:** "The second account is not reading the first user's private collection. Both authorized accounts subscribe to their household paths. Firestore streams notify each repository, and the UI rebuilds. If the account or membership changes, old streams are cancelled and version checks reject their late callbacks."
- **Question:** Why test two different accounts instead of one account on two phones? **Answer:** Different UIDs demonstrate household authorization and separate read states; the same UID would not establish account isolation.

O4. Atomic shared activity and personal read receipts

- **Files/symbols:** `lib/features/pantry/data/pantry_store.dart:8-48`, `PantryStore.write`; `lib/core/services/household_activity_store.dart:16-57,78-174`, `HouseholdActivityStore.publish/markRead/watch`; `lib/core/services/notification_service.dart:63-91,131-189,510-555`, merged feed and `markRead/markAllRead`; `lib/core/models/app_notification.dart:50-55`, type display label; `lib/core/widgets/app_shell.dart:88-130`, unread badge.
- **Trigger/flow:** Pantry edit writes stock plus a household activity event atomically → the member's activity stream merges with their receipt stream → inbox and badge rebuild → opening stores only that user's read receipt.
- **Validation/data:** Event type is `pantryItemUpdated`, with `includeSelf:false`. The receiving member sees title **Pantry**, body **A household member updated pantry stock.** Receipts are `users/{uid}/notificationReceipts/household:{familyId}:{eventId}`. Events before a member's `joinedAt` are filtered out.
- **Loading/failure:** Shared feed waits for both event and receipt data, exposes errors and can retry. Do not promise an event for every possible operation: inspected grocery checking and join/role paths do not create the same activity. Manual grocery activity is best-effort, unlike the atomic pantry event.
- **Say aloud:** "We store the shared event once under the household, then keep each user's read receipt separately. That avoids writing into every other user's private notification collection. The pantry update and event are atomic, and the actor is excluded. Reading the event on one account does not mark it read for everyone."
- **Question:** Why did the owner not receive their own update? **Answer:** Pantry activity explicitly excludes its actor; use the other member's inbox to demonstrate reception.

O5. Notification navigation and native-push limits

- **Files/symbols:** `lib/features/notifications/pages/notifications_page.dart:18-45,69-104`, `NotificationsPage/NotificationInbox._run`; `lib/core/routing/notification_route.dart:6-74`, `notificationRoute`; `lib/features/notifications/pages/notification_target_page.dart:9-98`, `NotificationTargetPage`; `lib/core/services/notification_service.dart:197-243,319-395`, lifecycle/token/feed handling; `lib/main.dart:31-34,157-160,230-299`, background entry and navigation setup.

- **Trigger/flow:** Tap card → awaited `markRead` → type-specific route. Pantry/grocery events open tabs. Meal events resolve an authorized loaded plan/meal target rather than manufacturing a missing draft. Native FCM interactions enter a separate authenticated/session-checked navigation path.
- **Validation/data:** Pending actions capture account/session context. Deleted, inaccessible or ambiguous meal targets show a recovery state, not another user's meal. Release App Check uses Play Integrity/DeviceCheck; debug/test paths are separate, and the obsolete Android release override is removed.
- **Loading/failure:** Inbox action errors display feedback; mark-all-read affects the current user. In-app delivery and Android background push are different systems. A trusted sender and end-to-end physical-device acceptance are not established merely by having `firebase_messaging` code.
- **Say aloud:** "Tapping an alert first updates this account's read state, then routes using the event type and authorized target. If the meal is gone or the account changed, it shows recovery instead of opening a guessed item. Our demonstration proves only the in-app actions we actually observe, not background push delivery."
- **Question:** Do household notifications require Cloud Functions here? **Answer:** The demonstrated in-app activity/receipt feed does not. Native remote push would require a trusted sender; no Functions deployment or billing change is claimed.

Important honest answers and verification boundaries

- **"Did you test two physical phones?"** The coding agent did not. Report only the presenters' actual preflight observations as user-verified; distinguish those from emulator and automated results.
- **"Does every provider/model work?"** Adapters and registry validation are implemented. The retained live evidence covers a bounded Gemini model workflow, not every selectable model or account's quota.
- **"Can you demonstrate everything offline?"** No. Cached data can help, but live generation, account actions, shared server confirmation and many writes require connectivity.
- **"Is the app production-ready?"** This is an academic prototype. Shared APK credentials are extractable, private provider storage is not a hardware vault, the cooperative quota is not authoritative anti-abuse protection, and native push/physical multi-device acceptance remain limited.
- **"Was the old exposed App Check secret revoked?"** No revocation is established. The owner explicitly accepted that existing Git-history risk for the source push. Removing a debug override from current code does not revoke a registered credential. Never reveal the value in a defense.
- **"Did these documents fix code or rebuild the APK?"** No. This task inspected source and created presentation documentation only.

Features deliberately not performed in the ten-minute live sequence

Fresh account registration/email delivery, password reset/change, invitation redemption, membership removal/rejoining, ownership transfer, archival/deletion, new AI replacement, provider import/switching, destructive grocery clearing, permission-denial recovery, offline restart and native background push are explained here or rehearsed separately. They are omitted from the timer because they add waiting, risk or duplicate coverage - not because their mere presence in code proves them working.

4. WhatMeal feature-to-code index

Paths are relative to the repository root. Symbols were checked against current source for the October 29, 2026 documentation. Detailed ranges and execution flows are in CODE_DEFENSE_GUIDE.md. “Live” below means an action planned in the timed script, **not** an agent-verified physical-device pass.

Feature	UI file	Logic/service file	Model/data file	Key method	Presenter
Bootstrap and state injection	`lib/main.dart`, `lib/app.dart`	`lib/core/routing/app_router.dart`	`lib/features/auth/data/auth_repository.dart`	`main`, `createAppRouter`	Jabez
Five-tab navigation	`lib/core/widgets/app_shell.dart`	`lib/core/routing/app_router.dart`	`lib/app.dart`	`AppShell.build`, `StatefulNavigationShell.goBranch`	Jabez
Login/register/email verification	`lib/features/auth/pages/login_page.dart`, `lib/features/auth/pages/register_page.dart`, `lib/features/auth/pages/verify_email_page.dart`	`lib/features/auth/data/auth_repository.dart`	`lib/features/auth/models/user_model.dart`	`_handleLogin`, `_handleRegister`, `login`, `register`, `reloadUser`	Jabez
Account edit/password change	`lib/features/profile/widgets/edit_profile_sheet.dart`	`lib/features/auth/data/auth_repository.dart`	`lib/features/auth/models/user_model.dart`	`_save`, `updateProfile`, `updatePassword`	Jabez
Planned-for dietary profiles	`lib/features/profile/pages/profile_page.dart`, `lib/features/profile/widgets/add_edit_family_member_sheet.dart`	`lib/features/profile/data/family_member_repository.dart`	`lib/features/profile/models/family_member.dart`	`_submit`, `addMember`, `updateMember`	Jabez
Manual plan and dish editing	`lib/features/meal_plan/pages/meal_plan_page.dart`, `lib/features/meal_plan/pages/generated_meal_plan_page.dart`	`lib/features/meal_plan/data/saved_plans_repository.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`, `lib/features/meal_plan/models/meal_plan.dart`	`_createCustomMealPlan`, `_showEditMealDialog`, `mealsForSaving`, `savePlan`	Jabez
Meal calendar, overlapping schedules	`lib/features/meal_plan/pages/meal_plan_calendar_page.dart`	`lib/core/routing/meal_route.dart`, `lib/core/utils/week_calendar.dart`	`lib/features/meal_plan/models/calendar_plan_day.dart`	`_select`, `_moveMonth`, `calendarPlansForDay`, `dateForMeal`	Jabez

Feature	UI file	Logic/service file	Model/data file	Key method	Presenter
Home status shortcuts/calendar view	`lib/features/home/pages/home_page.dart`	`lib/features/home/models/home_meals.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`	`_weekStat`, `matchesMealFilter`	Jabez
Saved-recipe recommendations	`lib/features/home/pages/home_page.dart`	`lib/features/home/models/home_meals.dart`	`lib/features/profile/models/family_member.dart`, `lib/features/meal_plan/models/saved_meal_plan.dart`	`homeRecommendations`	Jabez
Matching image/fallback	`lib/features/meal/widgets/meal_network_image.dart`	`lib/core/services/unsplash_service.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`	`fetchMealImage`, `dishMatches`, `_resolve`	Jabez
Provider/model/BYOK settings	`lib/features/ai_settings/pages/ai_settings_page.dart`	`lib/features/ai_settings/data/ai_provider_repository.dart`, `lib/core/services/ai/ai_provider_factory.dart`	`lib/core/models/llm_config.dart`, `lib/core/services/ai/model_registry.dart`	`selectProvider`, `updateModel`, `removeKey`, `validate`	Leonard
AI plan/context/quotas	`lib/features/meal_plan/pages/meal_plan_page.dart`	`lib/core/services/llm_service.dart`, `lib/core/services/ai_quota_store.dart`, `lib/features/meal_plan/data/generation_inventory.dart`	`lib/core/services/ai/ai_models.dart`	`_generateMealPlan`, `generateMealPlan`, `capture`, `buildPrompt`	Leonard
Gemini protocol/parser/errors	`lib/features/meal_plan/pages/meal_plan_page.dart`	`lib/core/services/ai/adapters/gemini_adapter.dart`, `lib/core/services/ai/ai_json_parser.dart`, `lib/core/services/ai/generation_failure.dart`	`lib/core/services/ai/ai_models.dart`	`_generate`, `parseMealList`, `validateMeals`, `validateRecipe`	Leonard
Explicit plan save/reopen	`lib/features/meal_plan/pages/generated_meal_plan_page.dart`, `lib/features/meal_plan/pages/saved_meal_plans_page.dart`	`lib/features/meal_plan/data/saved_plans_repository.dart`, `lib/features/meal_plan/data/saved_plan_stream.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`	`_saveDraft`, `savePlan`, `watchSavedPlans`	Leonard
Correct plan/meal detail identity	`lib/features/meal/pages/meal_detail_page.dart`	`lib/core/routing/meal_route.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`	`_resolveMeal`, `mealRoute`, `resolveSavedMeal`	Leonard

Feature	UI file	Logic/service file	Model/data file	Key method	Presenter
Favorites/history	`lib/features/meal/pages/meal_detail_page.dart`, `lib/features/meal_plan/pages/meal_history_page.dart`	`lib/features/meal_plan/data/saved_plans_repository.dart`, `lib/features/home/models/home_meals.dart`	`lib/features/meal/models/meal.dart`, `lib/features/meal_plan/models/saved_meal_plan.dart`	`toggleFavorite`, `matchesMealFilter`	Leonard
AI replacement and retained predecessor	`lib/features/meal/pages/meal_detail_page.dart`	`lib/features/meal_plan/data/saved_plans_repository.dart`, `lib/core/services/meal_replacement_store.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`	`_replaceWithAi`, `replaceMeal`, `replace`, `activeMeals`	Leonard
Saved plan reuse/deletion	`lib/features/meal_plan/pages/saved_meal_plans_page.dart`	`lib/features/meal_plan/data/saved_plans_repository.dart`	`lib/features/meal_plan/models/saved_meal_plan.dart`	`deletePlan`; reuse creates new draft/IDs	Leonard
Meal acceptance/completion	`lib/features/meal/pages/meal_detail_page.dart`, `lib/features/meal/widgets/meal_ingredients_panel.dart`	`lib/features/meal_plan/data/saved_plans_repository.dart`, `lib/core/services/meal_workflow_store.dart`	`lib/features/meal/models/meal.dart`	`_updateStatus`, `mealCookingProblem`, `updateMealStatus`, `setStatus`	Kyle
Pantry CRUD/expiry	`lib/features/pantry/pages/pantry_page.dart`, `lib/features/pantry/widgets/add_pantry_item_sheet.dart`	`lib/features/pantry/data/pantry_repository.dart`, `lib/features/pantry/data/pantry_store.dart`	`lib/features/pantry/models/pantry_item.dart`	`_editItem`, `_confirmDeleteItem`, `addItem`, `updateItem`, `write`	Kyle
Grocery shortages/units/provenance	`lib/features/grocery/pages/grocery_page.dart`	`lib/features/grocery/data/meal_grocery_projection.dart`, `lib/features/grocery/data/grocery_repository.dart`	`lib/features/grocery/models/grocery_item.dart`	`calculate`, `_publish`, `toItem`	Kyle
Manual grocery add/edit/dedup	`lib/features/grocery/widgets/grocery_item_editor.dart`	`lib/features/grocery/data/grocery_store.dart`	`lib/features/grocery/models/grocery_item.dart`	`normalizedName`, `addUnique`, `updateManual`	Kyle
Purchase marker and clipboard sharing	`lib/features/grocery/pages/grocery_page.dart`	`lib/features/grocery/data/grocery_repository.dart`	`lib/features/grocery/models/grocery_item.dart`	`setPurchased`, `cycleStatus`, `_shareGroceryList`	Kyle
Listener errors/retry/account scope	`lib/core/widgets/listener_recovery_notice.dart`	`lib/features/pantry/data/pantry_repository.dart`, `lib/features/grocery/data/grocery_repository.dart`	Repository `scopeVersion`, loading/error state	`retry`, `_resetListener`	Kyle

Feature	UI file	Logic/service file	Model/data file	Key method	Presenter
Household create/invite/join	`lib/features/family/pages/family_space_page.dart`, `lib/features/family/widgets/household_access_sheet.dart`	`lib/core/services/family_service.dart`, `lib/features/family/data/family_repository.dart`	`lib/core/models/family.dart`, `lib/features/family/models/family_space.dart`	`createFamily`, `createInvitation`, `joinFamily`, `joinSpaceWithKey`	Jobert
Roles, leave/removal/archive/ownership	`lib/features/family/pages/family_space_page.dart`, `lib/features/family/pages/household_dashboard_page.dart`	`lib/core/services/family_service.dart`, `firestore.rules`	`lib/features/family/models/family_space.dart`	`updateMemberRole`, `removeMember`, `deleteFamily`, `offerOwnership`, `acceptOwnership`	Jobert
Household real-time food sharing	`lib/features/pantry/pages/pantry_page.dart`, `lib/features/grocery/pages/grocery_page.dart`	`lib/features/family/data/family_repository.dart`, scoped food repositories	Canonical membership and scoped models	`_bindAccount`, `_listenToFamily`, `_listenToMembers`	Jobert
Shared inbox/read/unread state	`lib/features/notifications/pages/notifications_page.dart`, `lib/core/widgets/app_shell.dart`	`lib/core/services/household_activity_store.dart`, `lib/core/services/notification_service.dart`	`lib/core/models/app_notification.dart`	`watch`, `markRead`, `markAllRead`	Jobert
Notification routing/deleted targets	`lib/features/notifications/pages/notification_target_page.dart`	`lib/core/routing/notification_route.dart`, `lib/core/services/notification_service.dart`	`lib/core/models/app_notification.dart`	`notificationRoute`, `markRead`	Jobert
Release App Check/native messaging	`lib/main.dart`	`lib/core/services/ai/firebase_ai_app_check.dart`, `lib/core/services/notification_service.dart`	`lib/core/config/runtime_environment.dart`	`main`, `getLimitedUseToken`, `initialize`	Jobert

Extended trace: manual and AI are different entry paths

Manual: `_createCustomMealPlan` → `GeneratedPlanDraft` with empty authorable slots → editor `_showEditMealDialog` → `mealsForSaving` drops untouched slots → explicit `_saveDraft` → `SavedPlansRepository.savePlan` → Firestore plan plus nested meals → `watchSavedPlans` → calendar/details.

AI: `_generateMealPlan` → `GenerationInventory.capture` and synchronized settings → `LlmService.generateMealPlan` → `AiProviderFactory` → actual provider adapter → prompt/HTTP or retained SDK → JSON parser → meal/slot/restriction validation → result with actual provider/model → **unsaved** draft → explicit save path above.

Shared pantry/inbox: edit sheet → `PantryRepository.updateItem` → `PantryStore.write` transaction saves item and household event → member's pantry listener and `HouseholdActivityStore.watch` → rendered update/inbox → own receipt through `NotificationService.markRead` → `notificationRoute`.

Live coverage versus deliberate omissions

Demonstration treatment	Features
Timed live actions, subject to actual success	Manual author/edit/save; calendar; Home recommendations view; AI request with cutoff; saved plan reopen/details/favorite; meal acceptance; pantry edit; grocery quantity view/check/manual add; member view; a shared pantry update and inbox routing.
Conditional	New AI save, optional manual grocery addition, completion-ready meal, actual two-phone refresh/new inbox event. Use honest fallback if prerequisites or results are absent.
Prepared before timer	Install/launch on both devices; verified Auth accounts; household creation/invitation redemption; actual provider/model configuration; family profiles and saved backup.
Explained, not performed live	Password/account recovery; plan reuse/delete; new AI replacement; pantry add/delete; grocery edit/delete/clipboard share; Home compact/expanded control and extra history filters; membership removal/rejoining/role changes/ownership/archive; offline and listener-error recovery.
Not established as working by this documentation	Latest physical two-phone end-to-end acceptance; every provider/model live; background/cold-start native push; secure commercial shared-key distribution.

There is no implemented automatic pantry deduction on meal completion or automatic pantry stock addition from grocery checking. Expiration display/filter is implemented; a scheduled expiration push producer was not established. Do not present those as existing automation.

5. WhatMeal instructor Q&A

Twenty-four short, source-grounded answers: **six per presenter**. Study these separately; they do not add time to the ten-minute script. Each member should also know the architecture answer in question1 and the honesty rules at the end.

Jabez - application structure, identity and manual planning

1. What is the overall architecture?

"Flutter pages handle interaction, repositories expose changing application data, and services handle Firebase or AI operations. Models translate stored/request data into typed objects. GoRouter handles routes and authentication redirects. Provider injects some services, while many screens listen directly to ValueNotifier repositories."

Source: `lib/main.dart:214-220`; `lib/app.dart:8-59`; `lib/core/routing/app_router.dart:29-104`. Do not claim a uniform BLoC/Riverpod architecture.

2. Why does signing in not always open Home immediately?

"The router checks both authentication and email verification. An unsigned account goes to login; an authenticated but unverified account goes to the verification page. I've Verified My Email reloads the Firebase user before the app decides whether to open Home."

Source: `lib/core/routing/app_router.dart:34-56`; `lib/features/auth/pages/verify_email_page.dart:39-98`; `lib/features/auth/data/auth_repository.dart:83-99`.

3. Are family profiles the same as household accounts?

"No. A planned-for profile stores a person's age, preferences and restrictions. Household membership connects an authenticated UID to a household and a role. We can plan meals for someone who does not have an app account, but that does not give them permission to read the household."

Source: `lib/features/profile/models/family_member.dart:3-69`; `lib/features/profile/data/family_member_repository.dart:59-144`; `lib/features/family/models/family_space.dart:1-40`.

4. How do you stop a manual plan's blank template meals from being saved as real dishes?

"The manual creator makes authorable slots. Before saving, mealsForSaving omits untouched blank manual slots. A slot with partial content but no dish name is still invalid. The repository then writes the authored meals; opening an editor alone is not persistence."

Source: `lib/features/meal_plan/pages/meal_plan_page.dart:118-183`; `lib/features/meal_plan/models/saved_meal_plan.dart:365-385`; `lib/features/meal_plan/data/saved_plans_repository.dart:269-321`.

5. What determines the calendar date, especially across weeks/months?

"It uses the plan's scheduled start date and each meal's day index, not createdAt. The overview combines eligible saved plans for that day. Month navigation clamps the selected day to a valid day in the target month. The detail route also retains the owning plan path."

Source: `lib/features/meal_plan/models/calendar_plan_day.dart:7-46`; `lib/features/meal_plan/models/saved_meal_plan.dart:429-464`; `lib/features/meal_plan/pages/meal_plan_calendar_page.dart:39-90`.

6. Does Home generate a new recommendation with AI every time it opens?

"No. Home filters saved meals scheduled for today against current participants, recipe content and restrictions, then ranks favorites and preference matches. It shows a loading, empty or retry state when appropriate. AI generation is a separate action on the Meal Plan page."

Source: `lib/features/home/models/home_meals.dart:20-77`; `lib/features/home/pages/home_page.dart:487-573`.

Leonard - AI, parsing and saved plans

7. Are you using Firebase AI Logic or the Gemini Developer API?

"The adapter supports both. A personal Gemini key uses direct REST; the configured academic credential pool also uses the Developer API directly. Without either, the retained Firebase AI path uses Auth and App Check. We must describe the actual installed configuration rather than calling every request Firebase AI."

Source: `lib/core/services/ai/adapters/gemini_adapter.dart:43-76,164-193`. Do not reveal the configuration's key values or say every clone receives them.

8. What information is included in generation?

"The request includes selected people and ages, meal slots, cuisine, restrictions, preferences, pantry quantities, shopping context and optional budget. The page first requires current loaded inventory and synchronized provider settings. Shopping ingredients are described as not yet available pantry stock."

Source: `lib/features/meal_plan/data/generation_inventory.dart:22-40`; `lib/core/services/ai/ai_models.dart:64-158`; `lib/core/services/ai/pantry_prompt_context.dart:5-18`.

9. How do you handle malformed or incomplete AI output?

"The parser extracts and normalizes supported JSON forms, then validation checks the requested meal slots, title, quantities, ingredients and instructions. Restriction checks run as well. A successful HTTP response alone does not make a valid plan, and these checks are not a food-safety certification."

Source: `lib/core/services/ai/ai_json_parser.dart:12-80`; `lib/core/services/ai/ai_models.dart:38-59,349-358`; `lib/core/services/llm_service.dart:271-351`.

10. If AI succeeded but Firestore failed, must you generate again?

"No. Generation produces an unsaved draft. Explicit Save Plan writes it, and a failure keeps the editor available for a save retry. We do not call an unsaved editor a successful saved plan or discard it just to navigate away."

Source: `lib/features/meal_plan/pages/meal_plan_page.dart:353-387`; `lib/features/meal_plan/pages/generated_meal_plan_page.dart:51-96,301-445`; `lib/features/meal_plan/widgets/unsaved_plan_guard.dart:24-58`.

11. Why do you keep the full plan path as well as a meal ID?

"A meal ID alone may appear in another saved plan. The route carries its exact plan path, and resolution also checks the account session. Ambiguous or stale targets are rejected rather than selecting the first matching ID. Favorites and replacement use that same identity boundary."

Source: `lib/core/routing/meal_route.dart:19-51`; `lib/features/meal_plan/data/saved_plans_repository.dart:183-193,377-418`.

12. Can AI freeze the whole timed demonstration? Is free generation unlimited?

"The progress dialog cannot be cancelled and there is no whole-flow deadline, so we prepare the second phone with a saved fallback. Requests have provider limits and our cooperative default allowance is ten per day. We do not promise unlimited access, independent quotas for several keys, or a refund for every failed request."

Source: `lib/features/meal_plan/pages/meal_plan_page.dart:285-325`; `lib/core/services/ai/adapters/gemini_adapter.dart:251-299`; `lib/core/services/llm_service.dart:58,271-351`; `lib/core/services/ai_quota_store.dart:10-41`.

Kyle - pantry, quantities, shopping and lifecycle

13. How do you calculate what the user still needs to buy?

"The grocery projection combines ingredient requirements from accepted saved meals, groups compatible names and units, and subtracts usable pantry stock. It retains exact source meal paths. Unparseable amounts are flagged for review, so we do not invent a conversion just to show a number."

Source: `lib/features/grocery/data/meal_grocery_projection.dart:24-201`; `lib/features/grocery/data/grocery_repository.dart:102-166`.

14. Can you combine kilograms, grams, cups and packages automatically?

"Only supported compatible units are converted, such as kilograms to grams or liters to milliliters. The code does not guess a density for volume-to-mass or a package's contents. Incompatible units can remain separate, and unknown amounts display Check recipe quantity."

Source: `lib/features/grocery/data/meal_grocery_projection.dart:54-127,130-201`.

15. What happens when you check a grocery item or complete a meal?

"The grocery checkbox stores a shopping marker or receipt; it does not add pantry inventory. A meal's completion updates its lifecycle after pantry checks; it does not deduct stock. Those are current implementation limits, so we do not present either action as an automatic inventory transfer."

Source: `lib/features/grocery/data/grocery_repository.dart:343-381`; `lib/features/meal/widgets/meal_ingredients_panel.dart:9-26`; `lib/core/services/meal_workflow_store.dart:10-52`.

16. How do you avoid duplicate manual grocery items from two clients?

"Names are normalized for case and whitespace and mapped to deterministic document IDs. A transaction checks before creating the row. Current writers therefore coordinate on the same normalized name. Existing older random-ID duplicates are retained, and manual duplicate quantities are not automatically added together."

Source: `lib/features/grocery/data/grocery_store.dart:9-89`. Do not call this a migration of all legacy duplicates.

17. Does showing an expiration warning mean you send automatic expiry push reminders?

"No. The model computes days remaining from the recorded expiration date, and the pantry screen highlights or filters items within five days. We did not establish a scheduled expiration push producer. We can demonstrate that display without claiming a background notification was sent."

Source: `lib/features/pantry/models/pantry_item.dart:5-46`; `lib/features/pantry/pages/pantry_page.dart:18-89,490-529`.

18. Why not calculate groceries while pantry data is still loading?

"Unknown stock is not zero stock. The repositories expose dependency readiness and errors so the app can wait or offer Retry. Session and scope checks also reject delayed callbacks from a previous account. Treating a failed listener as an empty pantry would create misleading shopping requirements."

Source: `lib/features/grocery/data/grocery_repository.dart:102-129,188-298`; `lib/features/pantry/data/pantry_repository.dart:56-129`.

Jobert - membership, security, sharing and notifications

19. How does an invitation grant household access safely?

"It is an expiring, single-use token. Joining uses a transaction to consume the invitation and create the canonical member record plus account membership index together. Later access depends on stored membership and role rules, not on a client claiming a household ID or owner role."

Source: `lib/core/services/family_service.dart:37-46,93-170`; `firestore.rules:67-177`.

20. What is the difference between Firebase Auth, App Check and Firestore rules?

"Auth identifies the signed-in user. App Check attests the app making protected requests. Firestore rules decide what that user may read or write based on UID, stored membership, role and schema. Passing one layer is not permission to ignore the other layers."

Source: `lib/main.dart:148-160`; `lib/features/auth/data/auth_repository.dart:113-175`; `firestore.rules:207-241,352-401`.

21. Does the second phone read the owner's private documents?

"Authorized household accounts subscribe to shared collections under families, while personal collections stay under each user's UID. Joining switches the active scope; it does not copy all personal records. Listener cleanup and version checks protect account switching. We call synchronization verified only when we actually observe the second phone update."

Source: `lib/features/family/data/family_repository.dart:36-72,145-238`; `lib/features/pantry/data/pantry_repository.dart:62-129`; `lib/features/meal_plan/data/saved_plans_repository.dart:86-181`.

22. How can two members have different unread states for one event?

"The household event is stored once. Each account has its own receipt document under users. Opening the alert updates that user's receipt, so another member's unread state is not overwritten. Pantry updates exclude the actor and new members do not receive every event from before they joined."

Source: `lib/core/services/household_activity_store.dart:50-57,78-174`; `lib/core/services/notification_service.dart:510-555`.

23. Are these notifications real push messages when the app is closed?

"The demonstrated history is an in-app Firestore activity and receipt feed. FCM/background routing code also exists, but that does not prove trusted sending or closed-app delivery. We are not claiming native push passed on two physical phones or that Cloud Functions were deployed."

Source: `lib/core/services/notification_service.dart:243-395`; `lib/main.dart:31-34,230-299`; `lib/features/notifications/pages/notifications_page.dart:18-45`.

24. Is this prototype production-ready, and what is the unresolved security issue?

"We are presenting an academic prototype with clear limits. The configured APK's shared AI credentials are extractable, and an exposed historical App Check debug credential has not been confirmed revoked. The owner accepted that Git-history risk for publication. Removing debug-only code did not revoke it, and we must not reveal the value."

Source/evidence: `audit/final/ACADEMIC_RELEASE_20261026.md`; current release provider selection in `lib/main.dart:157-160`; the explicit owner decision recorded in the source commit message. Do not claim this is a completed security remediation.

Answers any member can safely use

- **If asked for a result you did not test:** "That path is implemented, but we have not verified that exact environment. Here is the code and our recorded evidence."
- **If live AI fails:** "This request did not complete successfully. We are switching to a previously saved plan, not presenting it as a new AI result."
- **If asked to show a credential:** "We can show the provider configuration flow and relevant code, but we will not display private keys, invitation secrets or debug tokens."
- **If asked for a code detail you cannot recall:** Open FEATURE_CODE_INDEX.md, locate the callback and follow the repository/service chain. Do not invent a method or claim a generic Flutter pattern that the app does not use.
- **If asked about verification:** Separate automated tests, Firebase emulator cases, the bounded host live-AI proof, prior Android-emulator runs and actual user-observed phone results. Only the last category establishes what happened on those physical phones.